

Fruit Classification

Wei Chen
Columbia University
wc2917@columbia.edu

Lamees Alwasil
Barnard College
laa2203@barnard.edu

Arth Singh
Columbia University
as7389@columbia.edu

Abstract—An accurate and efficient fruit classification is crucial in the modern management of the supermarket and agricultural sector. This study will focus on various computer vision and machine learning approaches to classify fruit images trained with public datasets. The trained model should be able to classify the fruits correctly under different lighting and noise conditions presented in a real-world environment. The model will be evaluated on the number of correctly identified images from unseen testing images. The desired outcome of the experiment should be a model whose computational efficiency (affordable under the majority of the environment) while retaining a high testing accuracy.

I. INTRODUCTION

Link to project code files: https://github.com/AnUnknownStranger/Fruit_Classification

A. Background and Motivation

Manual fruit sorting has been a time-consuming process in both the agricultural and supermarket sectors. In terms of operations, there are a huge number of workers are needed to inspect and classify each fruit. The process is not only slow but also inconsistent due to workers' personal judgment, and sometimes even causes errors. Therefore, developing an efficient fruit recognition system can significantly reduce the waste in human resources. This automated process will improve the general accuracy, accelerate checkout, packaging processes, and product placement. Additionally, the data for different types of fruits is widely available, making the development process cheap and scalable.

B. Objective

- Create a robust fruit detection system capable of deployment in varied, uncontrolled environments.
- The model should accurately classify various types of fruits with high precision and recall.
- The model should capture visual patterns from the training dataset, focusing on intrinsic features like shape and texture.
- The model should perform well under various lighting conditions and background noises

II. RELATED WORK

Fruit classification has garnered significant attention in computer vision research due to its practical applications in agriculture, retail automation, and quality control. Researchers have explored various approaches ranging from traditional feature extraction methods to deep learning architectures.

A. Traditional Feature-Based Methods

Early work in fruit classification primarily relied on hand-crafted features extracted from images. Color-based features, particularly those derived from HSV color space, have been widely used due to the distinctive color signatures of different fruit types. GarageFarm.net [4] provides a comprehensive analysis of color histogram techniques, demonstrating how color distribution patterns can effectively distinguish between fruit categories. However, color-based methods alone often struggle with variations in lighting conditions and intra-class color diversity, such as red versus green apples.

Shape and texture features have also been extensively studied. Pramono et al. [7] evaluate Histogram of Oriented Gradients (HOG) for carica classification, a specific papaya grown in the mountains, showing its effectiveness in capturing structural information to classify the different maturity levels of the papaya. Canny edge detection and HOG features capture shape and contour patterns, while Gray-Level Co-occurrence Matrix (GLCM) features characterize surface texture patterns, enabling discrimination between fruits with similar colors but different surface characteristics.

B. Machine Learning Classifiers

Traditional machine learning classifiers have been paired with handcrafted features for classification. Goel et al. [5] present a KNN-based methodology involving preprocessing, object recognition, feature extraction, and classification stages for agricultural applications. SVMs and Random Forest classifiers have also shown promising results when combined with color histograms and edge features. While computationally efficient, these methods rely heavily on domain expertise for feature engineering and struggle to capture complex visual patterns.

C. Deep Learning Approaches

The development of Convolutional Neural Networks (CNNs) revolutionized fruit classification by enabling automatic feature learning directly from raw pixel data. He et al. [3] introduced ResNet architecture with residual learning, which addresses the vanishing gradient problem in deep networks. ResNet-18 has become particularly popular for fruit classification due to its balance between model depth and computational efficiency. The use of transfer learning from ImageNet-pretrained models has further enhanced performance, allowing models to leverage learned representations from millions of images.

Naranjo-Torres et al. [9] provide a comprehensive review of CNN architectures applied to fruit image processing, analyzing various models including AlexNet, VGGNet, GoogLeNet, and ResNet families. Their survey demonstrates how CNNs have progressively improved fruit classification accuracy through deeper architectures, skip connections, and transfer learning strategies. Zawbaa et al. [10] compared FNNs with traditional approaches, demonstrating that CNNs can learn multi-scale features that capture both local texture patterns and global shape information.

While some researchers have explored combining handcrafted features with machine learning classifiers, deep learning approaches have increasingly demonstrated superior performance by automatically learning optimal feature representations, eliminating the need for manual feature engineering.

D. Data Augmentation and Real-World Applications

Dataset quality and size have been identified as critical factors in model performance. Rocha et al. [8] investigate data augmentation techniques, showing how synthetic variations including rotation, scaling, and color jittering can improve model robustness and generalization. Their work also explores fruit quality assessment, such as ripeness detection, extending classification beyond mere identification to practical agricultural applications.

Mamat et al. [6] explore real-world applications of fruit detection systems, highlighting the versatility of deep learning approaches across different agricultural and commercial contexts. Zhang et al. [11] discuss applications for automated fruit counting, quality assessment, and yield prediction in orchards, emphasizing practical challenges including occlusion, varying maturity stages, and different weather conditions.

E. Research Gaps and Contributions

The literature demonstrates a clear progression from traditional computer vision techniques toward deep learning-based solutions, with ResNet architectures emerging as particularly effective for fruit classification tasks. However, several critical gaps remain in existing research. First, most studies focus solely on maximizing classification accuracy without systematically analyzing the trade-off between dataset size and computational efficiency. Second, comprehensive head-to-head comparisons between traditional feature-based methods and deep learning approaches under controlled experimental conditions are limited. Third, the optimal dataset size that balances model performance with training cost remains underexplored, particularly regarding the point of diminishing returns.

This study addresses these gaps through systematic experimentation that evaluates both handcrafted features (HSV color histograms, edge detection, and texture features) and ResNet-18 across varying dataset sizes. By analyzing accuracy, training time, and computational requirements, we provide practical insights for deploying efficient fruit classification systems in resource-constrained environments such as agricultural settings and retail automation systems.

III. METHOD OVERVIEW

A. Data Loading and Preprocessing

The two major datasets we used for this experiment were taken from Kaggle.

1. Fruits Classification (<https://www.kaggle.com/datasets/utkarshsaxenadn/fruits-classification>)

- **Total Images:** 10,000 labeled fruit images.
- **Classes:** 5 distinct fruit categories (Apples, Bananas, Grapes, Mangoes, and Strawberries).

2. Fruit Recognition (<https://www.kaggle.com/datasets/chrisfilo/fruit-recognition>)

- **Total Images:** 44,406 labeled fruit images.
- **Classes:** 15 distinct fruit categories (Apple, Banana, Carambola, Guava, and many others).

Each of these images was converted into numpy pixels while assigning them their corresponding labels. We have normalized each image by simply dividing it by a max pixel value of 255 for faster convergence in deep learning models. Finally, we have resized each image to 224×224 for data consistency.

B. HSV Color Histogram

The first strategy we explored is a combination of HSV (Hue, Saturation, Value) color histogram and a support vector machine to perform classification. This method is initiated through how fruits are highly distinguishable by their color. For example, apples have a dominant red color, bananas have a yellow color, and grapes have a purple color. This model can work extremely well while retaining a low cost compared to many other LLMs.

C. Edge Detection

The second strategy that we explored is edge detection, using both Canny operators and HOG (Histogram Oriented Gradient) to classify the fruits based on their shapes. Edge detection highlights the boundaries and contours of each fruit, giving us a structural outline that helps us distinguish between the classes. Canny identifies sharp intensity changes, giving a clean binary edge map, while HOG captures the distribution of gradient orientation across the image. This means that it is easier to distinguish fruits with distinct geometric. For instance, a banana has an elongated shape compared to an apple that is round, making each fruit a lot easier to classify from their structural characteristics.

D. Texture Detection

Another strategy that we explored is texture detection, using the Gray-Level Co-occurrence Matrix (GLCM) to classify fruits based on the patterns present on their surfaces. GLCM measures how often pairs of pixel intensities occur at specific spatial relationships, allowing us to capture key texture characteristics of each fruit. From these matrices, we extract statistical features such as contrast, homogeneity, energy, and correlation, which describe the smoothness or irregularity of a surface. These texture-based cues help distinguish fruits that

may share similar shapes but differ in surface patterns. For example, the smooth appearance of an apple compared to the more uneven or speckled texture of strawberries. By converting these texture properties into feature vectors, GLCM provides a structured way to classify fruits using surface information.

E. HSV Color Histogram & Edge Detection

In addition to using HSV color histogram and edge detection independently, we also explored a combined approach that integrates the two feature types together. HSV histograms capture the overall color distribution of the fruit, providing a strong cue for distinguishing the fruits based on their color, but some fruits have different the same colors, so we combined it with Canny and HOG to highlight the structural information of the fruits. By merging the two features together, it allows us to leverage more information, such as, color helps separate fruits with similar shapes but different pigmentation, and edges help distinguish fruits with similar colors but different geometries. This combined approach helps create a more comprehensive characterization of each fruit, improving the robustness and accuracy of the classification.

F. ResNet-18

Eventually, this brings us to Convolutional Neural Network (CNN), one of the common strategies for the fruit classification problem. CNN is known for its ability to automatically learns spatial features directly from the raw pixel data by applying convolutional filters and pooling layers for dimensionality reduction, making it extremely effective in image classification problems.

ResNet-18 is one of the effective CNN architectures that stood out. ResNet-18 gives us a base pretrained model that has already been trained with ImageNet-1K data, which stores over 1.28 million images. This has made it an effective image classification model even before fine-tuning on a specialized fruit dataset. Moreover, ResNet-18 has shallower layers(only 18 layers) compared to ResNet-50/101, making it efficient, faster to train, and converges on smaller datasets while still capturing the feature patterns. This architecture has gained popularity due to its balance of performance and computational efficiency. Since fruits tend not to have extremely complex features, 18 layers should be sufficient in recognizing patterns from fruit images. Furthermore, ResNet-18 uses residual learning to avoid the vanishing gradient problem, enabling the deeper networks to be trained without significant degradation.

IV. EXPERIMENT AND RESULT

A. HSV Color Histogram Experiment

- RGB Images are loaded as testing, training, and validation sets with a size of 224×224 .
- Each image is transformed into HSV color space that has a 32-bin histogram for each of the H,S,and V channels.
- Concatenate the H,S,and V histogram from each image and apply L1 normalization.

- An Support Vector Machine classifier is built using a pipeline of StandardScaler (with_mean = false) and an RBF-kernel SVM.
- Apply 5-fold cross-validation Grid Search to find the best performing C and gamma hyperparameters:
 - C: {1, 3, 10, 30, 100}
 - gamma: {scale, auto, 10^{-3} , 3×10^{-3} , 10^{-2} , 3×10^{-2} , 10^{-1} }
- Evaluate the best model on training, validation, and testing sets with a metrics of accuracy, F1-score, Recall, and Precision

B. HSV Color Histogram Result

[train] accuracy=0.7639 macroF1=0.7641					
[valid] accuracy=0.5350 macroF1=0.5285					
[test] accuracy=0.4900 macroF1=0.4887					
		precision	recall	f1-score	support
	Apple	0.39	0.35	0.37	20
	Banana	0.41	0.55	0.47	20
	Grape	0.62	0.65	0.63	20
	Mango	0.53	0.45	0.49	20
	Strawberry	0.53	0.45	0.49	20
	accuracy			0.49	100
	macro avg	0.49	0.49	0.49	100
	weighted avg	0.49	0.49	0.49	100

Fig. 1: Results for HSV Color Histogram training

C. Edge Detection Experiment

Data Preparation:

- Similarly, RGB images are loaded into training, testing, and validation sets, each resized to 224×224 .
- The loaded training images are split into 80% train and 20% test
- Each item is converted into grayscale for edge detection

Canny Edge Detection

- Apply Gaussian Smoothing to reduce noise
- Use the Canny edge detector with tuned lower and upper thresholds to extract binary edge maps.
- Flatten the binary edge map into a feature vector for each image.

HOG Edge Detection

- Compute the Histogram of Oriented Gradients (HOG) descriptors for each image using:
 - Fixed cell size
 - Block normalization
 - 9 Orientation bins
- Flatten the HOG descriptors into a feature vector representing the fruit's shape and contour structure.

Classifier: a Random Forest is trained separately for each feature type: $n_estimators=100$, $max_depth=10$, $min_sample_split=5$, $min_sample_leaf=5$, $random_state=42$

Evaluation: similarly, evaluate the best model on training, validation and testing sets with a metrics of Accuracy, F1-score, Recall, and Precision.

D. Edge Detection Result

```

=====
Testing Canny...
=====
[train] accuracy=0.8113 macroF1=0.8147
[test] accuracy=0.3050 macroF1=0.2991
      precision    recall  f1-score   support

   Apple      0.36      0.23      0.28         40
  Banana      0.33      0.23      0.27         40
   Grape      0.22      0.25      0.24         40
   Mango      0.40      0.30      0.34         40
 Strawberry    0.29      0.53      0.37         40

 accuracy                   0.30         200
 macro avg      0.32      0.30      0.30         200
 weighted avg   0.32      0.30      0.30         200

=====
Testing HOG...
=====
[train] accuracy=0.9962 macroF1=0.9963
[test] accuracy=0.3000 macroF1=0.2880
      precision    recall  f1-score   support

   Apple      0.14      0.10      0.12         40
  Banana      0.38      0.50      0.43         40
   Grape      0.30      0.38      0.33         40
   Mango      0.33      0.33      0.33         40
 Strawberry    0.26      0.20      0.23         40

 accuracy                   0.30         200
 macro avg      0.28      0.30      0.29         200
 weighted avg   0.28      0.30      0.29         200

```

Fig. 2: Results for Canny and HOG training

E. Texture Detection Experiment

Data Preparation: similarly, RGB images are loaded into training, testing, and validation sets, each resized to 224×224 and the loaded training images are split into 80% train and 20% test.

GLCM Feature Extraction

- Convert the RGB images into grayscale.
- Quantize grayscale intensities from 0–255 $\rightarrow$ 0–15 (divide by 16) to reduce computation ($levels = 16$)
- Compute GLCM with:
 - Distance: $\{1, 2\}$
 - Orientation: $0, \frac{\pi}{4}, \frac{\pi}{3},$ and $\frac{3\pi}{4}$
 - symmetric = True, normed = True
- For each (distance, angle) pair, extract 5 Haralick properties: contrast, dissimilarity, homogeneity, energy, correlation.
- Concatenate all feature into a single vector 2 distances $\times$ 4 angles $\times$ 5 props = **40 features**).

Classifier: similarly to Edge Detection, a Random Forest is trained for each feature type.

Evaluation: evaluate the best model on training, validation and testing sets with a metrics of Accuracy, F1-score, Recall, and Precision.

F. Texture Detection Result

```

=====
Testing GLCM...
=====
[train] accuracy=0.9375 macroF1=0.9377
[test] accuracy=0.3350 macroF1=0.3299
      precision    recall  f1-score   support

   Apple      0.42      0.33      0.37         40
  Banana      0.44      0.38      0.41         40
   Grape      0.18      0.17      0.18         40
   Mango      0.37      0.25      0.30         40
 Strawberry    0.31      0.55      0.40         40

 accuracy                   0.34         200
 macro avg      0.35      0.34      0.33         200
 weighted avg   0.35      0.34      0.33         200

```

Fig. 3: Results for Texture Detection training

G. HSV Histogram & Edge Detection Experiment

Data Preparation: similarly, RGB images are loaded into training, testing, and validation sets, each resized to 224×224 and the loaded training images are split into 80% train and 20% test.

Feature Fusion

- **HSV & Canny:** concatenate HSV-histogram features with Canny features.
- **HSV & HOG:** concatenate HSV-histogram features with HOG features.

Classifier: a Random Forest classifier is trained separately for each combined feature using: $n_estimators = 100$, $random_state = 42$

Evaluation: Report performance on the test set using Accuracy and Macro F1-score.

H. HSV Histogram & Edge Detection Result

```

Results:
Accuracy (HSV + HOG): 0.4150
F1 Score (HSV + HOG): 0.4142
Accuracy (HSV + Canny): 0.5650
F1 Score (HSV + Canny): 0.5611

```

Fig. 4: Results for HSV Color Histogram combined with Edge Detection

I. ResNet-18 Experiment on Fruit Classification Dataset

Using just Dataset 1(Fruit Classification)

- Load and preprocess the training, testing, and validation datasets into Numpy Arrays.
- Each image is converted into PyTorch tensors while normalizing each image using ImageNet mean and standard deviation, making sure that it's compatible with pretrained ResNet-18.
- Load the pretrained ResNet-18 model while modifying the final fully connected layer with an output size equals to the number of fruit classes.
- Use an Adam Optimizer with a learning rate of $3e-4$ and a cross-entropy loss

- Train the model over 10 epochs while computing the forward pass, backward pass, training time, and training loss.
- Within each epoch, evaluate the accuracy on the training and validation set while checkpointing the model that has the best validation accuracy.
- Evaluate the best performing model on the testing set and analyze the accuracy for model generalization.

J. ResNet-18 Result on Fruit Classification Dataset

```

Loading train dataset...
train: Loaded 9700 images.
Loading valid dataset...
valid: Loaded 200 images.
Loading test dataset...
test: Loaded 100 images.

```

Fig. 5: Size of Datasets

```

Epoch 1/10 train_loss=0.4425 train_acc=0.8518 val_acc=0.8750 time=16.3s
-> saved best model: model.pth
Epoch 2/10 train_loss=0.2298 train_acc=0.9210 val_acc=0.9150 time=15.4s
-> saved best model: model.pth
Epoch 3/10 train_loss=0.1780 train_acc=0.9389 val_acc=0.8650 time=15.2s
Epoch 4/10 train_loss=0.1088 train_acc=0.9621 val_acc=0.8700 time=15.3s
Epoch 5/10 train_loss=0.0926 train_acc=0.9706 val_acc=0.9500 time=15.3s
-> saved best model: model.pth
Epoch 6/10 train_loss=0.0930 train_acc=0.9680 val_acc=0.9350 time=15.1s
Epoch 7/10 train_loss=0.0594 train_acc=0.9801 val_acc=0.9250 time=15.2s
Epoch 8/10 train_loss=0.1048 train_acc=0.9662 val_acc=0.8700 time=15.9s
Epoch 9/10 train_loss=0.0874 train_acc=0.9712 val_acc=0.9050 time=15.7s
Epoch 10/10 train_loss=0.0458 train_acc=0.9853 val_acc=0.9050 time=16.3s
Test accuracy: 0.9300 (classes: ['Apple', 'Banana', 'Grape', 'Mango', 'Strawberry'])

```

Fig. 6: Results for ResNet-18 Model on Fruit Classification Dataset

K. ResNet-18 Experiment on Fruit Classification Dataset and Fruit Recognition Dataset

The experiment will be conducted based on the same procedure as Part C, with additional Dataset 2(Fruit Recognition). We'll first split the whole dataset into training, testing, and validation sets. Add the dataset to the existing Fruit Classification Data and expand the types of fruits from 5 to 16. Finally, execute the procedure from part c and evaluate under the same metrics.

L. ResNet-18 Result on Fruit Classification Dataset and Fruit Recognition Dataset

```

Loading train dataset...
train: Loaded 54764 images.
Loading valid dataset...
valid: Loaded 5828 images.
Loading test dataset...
test: Loaded 5745 images.

```

Fig. 7: Size of Datasets

```

Epoch 1/10 train_loss=0.1425 train_acc=0.9535 val_acc=0.9940 time=393.1s
-> saved best model: model.pth
Epoch 2/10 train_loss=0.0765 train_acc=0.9746 val_acc=0.9940 time=259.2s
Epoch 3/10 train_loss=0.0613 train_acc=0.9796 val_acc=0.9966 time=243.6s
-> saved best model: model.pth
Epoch 4/10 train_loss=0.0452 train_acc=0.9851 val_acc=0.9938 time=236.8s
Epoch 5/10 train_loss=0.0360 train_acc=0.9879 val_acc=0.9969 time=206.5s
-> saved best model: model.pth
Epoch 6/10 train_loss=0.0312 train_acc=0.9896 val_acc=0.9959 time=234.3s
Epoch 7/10 train_loss=0.0247 train_acc=0.9920 val_acc=0.9949 time=217.2s
Epoch 8/10 train_loss=0.0214 train_acc=0.9932 val_acc=0.9935 time=223.6s
Epoch 9/10 train_loss=0.0200 train_acc=0.9933 val_acc=0.9964 time=219.3s
Epoch 10/10 train_loss=0.0168 train_acc=0.9947 val_acc=0.9961 time=216.9s
Test accuracy: 0.9981 (classes: ['Apple', 'Banana', 'Carambola', 'Grape', 'Guava',
'Pomegranate', 'Strawberry', 'Tomatoes', 'muskmelon'])

```

Fig. 8: Results for ResNet-18 Model on Fruit Classification and Fruit Recognition Dataset

classes = [Apple, Banana, Carambola, Grape, Guavam, Kiwi, Mango, Orange, Peach, Pear, Persimmon, Pitaya, Plum, Pomegranate, Strawberry, Tomatoes, Muskmelon]

M. ResNet-18 Experiment on Fruit Classification Dataset and Fruit Recognition Dataset trained partial data

Finally, we'll evaluate the model using different percentages of the available dataset while keeping all training conditions, hyperparameters, and environment settings constant. This allows us to analyze how the size of the dataset affects the model performance and discover the tradeoff between training cost and testing accuracy. We aim to estimate the optimal data size that has minimal impact on the overall accuracy while reducing the general computation cost.

N. ResNet-18 Result on Fruit Classification Dataset and Fruit Recognition Dataset trained partial data



Fig. 9: Comparison in the testing accuracy vs. Percentage of Training Data and training time vs. the percentage of training data

V. DISCUSSION

A. HSV Color Histogram

According to the results of the experiment, the model achieved approximately 76% training accuracy, 53.5% validation accuracy, and 49% testing accuracy, with macro F1-scores following a similar pattern. While the training performance is reasonable for a simple histogram-based classification approach, the lower validation and testing accuracies indicate that the model is overfitting and that this method is unlikely to generalize well for this dataset.

After further investigation into the issue, we have noticed several factors that have contributed to the underperformance of the model. First, the model is only capturing colors. This indicates that the model has limited ability in capturing the shape and texture of the fruits, resulted in an bad accuracy. Most importantly, the same fruits in the dataset tend to appear in highly distinguishable colors.



(a) Red Apple from Fruit Classification Dataset



(b) Green Apple from Fruit Classification Dataset

Fig. 10: Example fruit images from the dataset

Moreover, the images contain a large amount of distracting background noise including hands and different daily life elements.



(a) Apple from Fruit Classification Dataset



(b) Apple from Fruit Classification Dataset

Fig. 11: Example fruit images from the dataset

B. Edge Detection

This experiment evaluated whether shape/structure cues could improve fruit recognition beyond color alone. In fact, during edge detection the images were converted into grayscale, eliminating the color feature completely. Two edge-based representations were tested with the same pipeline.

The Canny feature vector was extremely compact, where there were only 5 values for edge density and Sobel magnitude statistics. This made it fast and less prone to overfitting, but it also limited discriminative power. Many fruit classes could

have very similar global edge statistics; for example, an apple could have similar edge statistics as a mango. This happened in particular when the background contained strong edges (e.g., cluster, hand) which distorted edge density and gradients.

HOG features encoded a richer distribution of local gradient orientations across the image. This typically captured shape and contour patterns more effectively than global edge statistics, so it was expected to generalize better when fruits differed by form (e.g., elongated vs round) even under moderate illumination changes. However, HOG could still be affected by the background since extraction was performed on the whole image.



(a) Mango from Fruit Classification Dataset



(b) Mango from Fruit Classification Dataset

Fig. 12: Example fruit images from the dataset

A key limitation with edge detection was that there were different ways to show the same fruit. For instance, in Figure 12, image (a) showed a whole mango and image (b) showed a mango smoothie; both of these images should have been classified as MANGO. However, since we used shape/structure for detection, this mango smoothie would fail to be classified as a mango since it did not fit into the standard edge detection pattern.

Another limitation to keep in mind was that this was tested on only train vs test, and the Random Forest hyperparameters were fixed. This meant that any observed train–test gap should be interpreted as generalization behavior under this single split rather than a fully tuned estimate.

C. Texture Detection

In this experiment, we focused on texture discrimination using GLCM (Haralick) features. GLCM features were useful when classes differed by surface patterns (smooth vs speckled vs textured). For example, a strawberry could be easily classified from its texture as it had a very specific speckled texture. This was because surface patterns summarized spatial co-occurrence statistics; however, they had practical weaknesses in this dataset setting:

- They were computed on the full image, so background texture could dominate the co-occurrence statistics.
- They were sensitive to changes in scale/pose and to how much of the fruit occupied the frame.
- They discarded color entirely since images were converted into grayscale.

Similar to the limitations with the previous two features, there were multiple different ways of illustrating the same fruit but in different forms. For example, the inside of an apple did not have the same texture as its exterior, making it more prone to fail through the training process.

D. HSV Histogram & Edge Detection

In this experiment, we tested whether combining complementary cues improved robustness: HSV histograms captured global color distribution, while edge detection captured structural information. The motivation for combining the two features together was that HSV alone could fail when fruits shared similar colors or when the same fruit appeared in different colors; adding shape cues helped reduce those confusions.

In our experiments, HSV + Canny outperformed HSV + HOG. A likely reason is that the Canny-based edge descriptor is very low-dimensional (HSV+Canny is 101 features) and captures coarse structural cues while acting as an implicit regularizer, making it less sensitive to background clutter. By comparison, HOG produces a much higher-dimensional representation (HSV+HOG is 1392 features), which can be harder to learn robustly with the fixed Random Forest settings and limited training samples, and may overfit to background gradients (e.g., edges from hands, containers, or image texture) rather than fruit structure.

E. ResNet-18

In this experiment, we have selected Adam as our optimizer instead of SGD, as Adam allows the model to converge faster than SGD while selecting a relatively low learning rate to avoid overshooting the minimum to balance the training stability and the training time. Together, since fruit patterns are simple, this model should be able to distinguish features effectively in a shorter amount of time without a greater issue of overfitting. Now according to the result of part c (ResNet-18 Experiment on Fruit Classification Dataset), we're able to reach a best result of 97% training accuracy, 95% validation accuracy, and 93% testing accuracy on 5 different types of fruits. This indicates that the model is able to effectively capture the visual pattern and learns the distinguishing features of each fruit class. The model can accurately recognize the fruit under various background noises and different lighting conditions. The minimal gap between the training, validation, and validation accuracy indicates that there is minimal overfitting in the pattern and generalizes well to the unseen data. Moreover, this simple experiment only requires around 9700 images for training, with an average training time of 15.5 seconds per epoch. This highlights the fact that ResNet-18 is computationally efficient, giving it greater scalability under various environments where computational resources

are limited. Therefore, this makes the model one of the best solutions to fruit classification and further investigation.

Building on these promising results, we further investigated the model by adding additional datasets on top of the existing ones to capture the fact that the supermarket and agricultural sectors have various types of fruits. Therefore, we made the choice to append the fruit recognition dataset on top of the existing dataset, expanding the types of fruits from 5 different types to 16 different types of fruits, and a dataset size of 10000 images to around 67000 images (54764 training images, 5828 validation images, and 5745 testing images). After executing the model training on the same computational environment, we're able to reach an optimal training accuracy of 98.8%, a validation accuracy of 99.7%, and a testing accuracy of 99.8%. This result indicates that there is minimal to no overfitting. The validation and testing accuracy have outperformed the training accuracy. The significant improvement in the performance compared to a smaller dataset highlights the importance of the dataset size and the diversity of the data. Additional data allows the model to learn more robustly and gives it a greater capability of capturing the fruit features under various noisy backgrounds and lighting conditions. Moreover, the shallow architecture of the ResNet-18 allows training and preprocessing to be done within roughly an hour compared to multiple hours or even days in some of the larger models. However, can we do better? Can we achieve a better performance without sacrificing too much accuracy while not using greater computational resources?

The answer to the previous questions is simply a yes, using one of the most effective and simple techniques: random sampling. We'll randomly select a percentage of the data from the full dataset and train the model with the selected subset. This allows us to analyze the model performance under different dataset sizes. One of the common issues we saw in a larger dataset is data redundancy, where many images tend to share a similar pattern.



(a) Apple from Fruit Recognition Dataset

(b) Apple from Fruit Recognition Dataset

Fig. 13: Similar pattern images with in large dataset

Training the model on such repetitive data can lead to overfitting, as the model learns the same feature over and over multiple times for every epoch, resulting in unnecessary training. Now we run into the issue of how to find the optimal sampling size without sacrificing too much accuracy. Since too much data might lead to overfitting, while too little data might result in underfitting. We have experimented with the model under 3 major percentage sampling of 10%, 20%, and 100%

with a respective of 5476 training images, 10952 training images, and 54764 training images. As a result:

- 10% of the dataset: achieved approximately 98.8% testing accuracy with a training time of 13 seconds per epoch.
- 20% of the dataset: improved to 99.5% testing accuracy with a training time of 23 seconds per epoch.
- 100% of the dataset: reached 99.8% testing accuracy, but the training time increased significantly to 250 seconds per epoch.

These results indicate that 20% of the dataset provides the optimal balance between accuracy and training time. We can sacrifice 0.3% of the overall accuracy with a roughly 10 times speed up in the training time. Anything lower than 20% accuracy will eventually result in underfitting, as shown by the decrease in testing accuracy while adding more data yields diminishing returns, with a rapid increase in the computation cost. This demonstrates how ResNet-18 can be fine-tuned with a smaller dataset while still receiving a desired outcome and the importance of careful dataset sampling for efficient model training.

VI. CONCLUSION

This study explored a range of computer vision techniques for fruit classification, with the goal of identifying an approach that balances classification accuracy, robustness, and computational efficiency in realistic deployment settings. Beginning with traditional feature-based methods and progressing toward deep learning models, we systematically evaluated how different visual cues, such as color, shape, texture, and learned hierarchical features, contribute to classification performance under varied lighting conditions and background noise. The experiments with handcrafted features, including HSV color histograms, edge-based descriptors, and texture features, demonstrated that these approaches can capture meaningful visual information at a relatively low computational cost. In particular, color-based representations proved effective when fruit classes exhibited consistent pigmentation. However, these methods struggled to generalize across variations in lighting, occlusion, and background clutter, as well as intra-class diversity such as differences in fruit ripeness or color. The resulting performance gap between training and testing sets indicates that relying on a limited set of handcrafted features is insufficient for robust fruit classification in unconstrained environments. In contrast, the ResNet-18 convolutional neural network consistently outperformed traditional methods across all evaluated metrics. Leveraging transfer learning from ImageNet allowed the model to learn rich and discriminative feature representations even with a relatively modest number of training images. The high and stable accuracy observed across training, validation, and testing sets suggests that ResNet-18 effectively captures both low-level and high-level visual patterns relevant to fruit classification, including color, shape, and texture, while remaining resilient to background noise and illumination changes. Expanding the training data by incorporating an additional fruit recognition

dataset further improved the model's performance, underscoring the importance of dataset size and diversity. The improved results indicate that a larger and more varied dataset enables the network to generalize more effectively by exposing it to a wider range of visual appearances for each fruit class. At the same time, the relatively shallow architecture of ResNet-18 ensured that this performance gain did not come at the cost of excessive training time or computational overhead, making it a practical choice for real-world applications. An important contribution of this work is the analysis of dataset size through controlled random sampling. The results show that training on a carefully selected subset of the data can achieve near-optimal accuracy while significantly reducing training time. In particular, using approximately 20% of the dataset, the results of this study suggest that deep learning approaches, and ResNet-18 in particular, offer a robust and scalable solution for automated fruit classification. While traditional feature-based methods remain useful as lightweight baselines, their limitations become pronounced in complex, real-world settings. Future work may explore more advanced data augmentation strategies, lightweight model compression, or real-time inference optimizations to further improve deployment feasibility. Additionally, extending the system to handle occluded fruits, mixed fruit scenes, or video-based inputs would be valuable directions for continued research.

VII. REFERENCES

REFERENCES

- [1] U. Saxena, "Fruits Classification," Kaggle, 2023. [Online]. Available: <https://www.kaggle.com/datasets/utkarshsaxenadn/fruits-classification>
- [2] C. Filo, "Fruit Recognition," Kaggle, 2023. [Online]. Available: <https://www.kaggle.com/datasets/chrisfilo/fruit-recognition>
- [3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. [Online]. Available: <https://arxiv.org/abs/1512.03385>
- [4] GarageFarm.net, "Color Histograms Demystified: From Theory to Practical Image Analysis," [Online]. Available: <https://garagefarm.net/blog/color-histograms-demystified-from-theory-to-practical-image-analysis>
- [5] N. Goel, P. Kaur, and S. Batra, "Fruit Classification Using K-Nearest Neighbor Algorithm," in *2023 International Conference on Device Intelligence, Computing and Communication Technologies (DICCT)*, 2023, pp. 596–600. [Online]. Available: <https://ieeexplore.ieee.org/document/10228051>
- [6] N. Mamat, M. F. Othman, R. Abdulghafor, M. A. Alwan, and Y. A. Gulzar, "Enhancing Image Annotation Technique of Fruit Classification Using a Deep Learning Approach," *Sustainability*, vol. 15, no. 2, p. 901, 2023. [Online]. Available: <https://www.mdpi.com/2071-1050/15/2/901>
- [7] A. Pramono et al., "Evaluation of Histogram of Oriented Gradient (HOG) and Learning Vector Quantization (LVQ) in Classification Carica Vasconcellea Cundinamarcensis," in *2019 International Conference on Information and Communications Technology (ICOIACT)*, 2019. [Online]. Available: <https://ieeexplore.ieee.org/document/8884314>
- [8] A. Rocha, D. C. Hauagge, J. Wainer, and S. Goldenstein, "Automatic Fruit and Vegetable Classification from Images," *Computers and Electronics in Agriculture*, vol. 70, no. 1, pp. 96–104, 2010. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S016816990900180X>
- [9] J. Naranjo-Torres, M. Mora, R. Hernández-García, R. J. Barrientos, C. Fredes, and A. Valenzuela, "A Review of Convolutional Neural Network Applied to Fruit Image Processing," *Applied Sciences*, vol. 10, no. 10, p. 3443, 2020. [Online]. Available: <https://www.mdpi.com/2076-3417/10/10/3443>

- [10] H. M. Zawbaa, M. Hazman, M. Abbass, and A. E. Hassanien, "Automatic Fruit Classification Using Random Forest Algorithm," in *2014 14th International Conference on Hybrid Intelligent Systems*, 2014, pp. 164–168. [Online]. Available: <https://ieeexplore.ieee.org/document/7086191>
- [11] B. Zhang, W. Huang, J. Li, C. Zhao, S. Fan, J. Wu, and C. Liu, "Principles, Developments and Applications of Computer Vision for External Quality Inspection of Fruits and Vegetables: A Review," *Food Research International*, vol. 62, pp. 326–343, 2014. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0963996914001707>